Software Lifecycle Control Standard - (SLCS)

Software Lifecycle Control Standard - (SLCS)

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Parent Standard

Governed Space: Software Lifecycle Control Governance

Standard ID: SLGA-SLCS-11

Origin ID: OOF-SLGA-SLCS

Version: 1.0

Status: Canonical Open

Effective Date: 15.10.2026

Compatibility: Methodology OS — Modular Compatibility

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: Protected Methodological Work

Canonical Language: UCL — Universal Canonical Language (English)

Canonical Definition

Software Lifecycle Control Standard (SLCS) is the eleventh Parent Standard within SLGA™ — Software Lifecycle Governance Architecture that governs how the lifecycle states, transitions, exceptions, authority relationships, control responsibilities, and governance conditions of identifiable software objects remain consistent, traceable, and controllable across the complete software lifecycle.

SLCS establishes software lifecycle control as a continuous, cross-cutting

governance function connecting the software's declared lifecycle state with its actual technical, operational, organizational, and evidentiary conditions.

The standard distinguishes lifecycle state identification from lifecycle

state validation, a proposed transition from an authorized transition, lifecycle activity completion from verified state achievement, and formal control assignment from effective control capability.

It requires material lifecycle states and transitions to remain associated

with identifiable software objects, applicable lifecycle requirements, supporting evidence, competent authority, responsible actors, operational conditions, unresolved obligations, and relevant governance decisions.

SLCS governs the consistency of lifecycle states across software

requirements, creation, dependencies, configuration, quality, security, release, deployment, change management, and retirement.

It recognizes that software does not always progress through a simple linear sequence. Software may return to earlier lifecycle activities, undergo parallel governance processes, remain in transitional states, experience exceptional conditions, or exist simultaneously in different states across versions, configurations, environments, and operational contexts.

SLCS therefore establishes mechanisms for maintaining lifecycle state

consistency, controlling transitions, governing exceptions, preserving control continuity, and assessing the effectiveness of lifecycle governance.

It does not replace the specialized Parent Standards responsible for governing individual lifecycle domains.

Instead, SLCS establishes the common lifecycle control conditions required

to preserve coherence between them.

Its principal governance output is the Software Lifecycle Control Governance Profile.

Operational Role

SLCS establishes the continuous control framework for software lifecycle

governance.

It governs:

• Software lifecycle identity

• Lifecycle state recognition

• Lifecycle state consistency

• Lifecycle state validation

• Lifecycle transition identification

• Lifecycle transition conditions

• Lifecycle transition authority

• Lifecycle transition verification

• Cross-standard governance continuity

• Lifecycle control responsibility

• Lifecycle authority alignment

• Lifecycle control capability

• Lifecycle control transfer

• Lifecycle control continuity

• Lifecycle exceptions

• Lifecycle deviations

• Lifecycle uncertainty

• Lifecycle conflicts

• Lifecycle re-entry consistency

• Lifecycle state reconciliation

• Lifecycle control evidence

• Automated lifecycle actions

• Autonomous software lifecycle governance

• Lifecycle control assessment

• Lifecycle control effectiveness

• Lifecycle control deficiencies

• Lifecycle reassessment

• Lifecycle governance outputs

SLCS establishes whether the software's lifecycle condition and governance

controls remain legitimate and consistent.

It does not independently perform the specialized activities governed by the other SLGA™ Parent Standards.

Core Question

How can software remain identifiable, controllable, and governable throughout its lifecycle when its states, transitions, actors, dependencies, operational conditions, authority relationships, and governance requirements continuously change?

Position in Architecture Governance Flow

SLGA™ — Software Lifecycle Governance Architecture

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-Cutting Governance Layer: Software Lifecycle Control

SLCS operates across all ten preceding Parent Standards.

It does not constitute an additional mandatory operational stage after retirement.

Instead, it governs the consistency, legitimacy, and continuity of

lifecycle control from initial software requirements through retirement and any continuing post-retirement obligations.

Its position is therefore cross-cutting rather than sequential.

• Governed Space

Software Lifecycle Control Governance

This governed space concerns the continuous governance of software

lifecycle states, transitions, exceptions, authority relationships, control responsibilities, and governance effectiveness.

It begins when an identifiable software object enters an applicable governed lifecycle context.

It remains applicable throughout creation, modification, validation,

release, deployment, operation, retirement, and continuing lifecycle obligations.

It includes the establishment of actual and declared lifecycle conditions,

verification of transition requirements, authority alignment, control continuity, exception governance, and cross-standard consistency.

It remains applicable when software enters an exceptional, uncertain, partially verified, or conflicting lifecycle condition.

The governed space does not end merely because a software object is declared retired where continuing lifecycle-related governance obligations remain.

Governed Object

The primary Governed Object is:

A Software Lifecycle Control Case representing the evidence-supported

relationship between an identifiable software object, its applicable lifecycle states, actual operational conditions, governing standards, authorized transitions, control responsibilities, exceptions, verification evidence, and resulting lifecycle governance status.

The governed object must preserve, where applicable:

Lifecycle Control Case ID

Software Object ID

Software Version

Configuration Reference

Environment Reference

Declared Lifecycle State

Verified Lifecycle State

Applicable Governance Standards

Lifecycle Transition Reference

Transition Authority

Control Responsibility

Control Capability

Applicable Exceptions

Unresolved Conditions

Supporting Evidence

Control Assessment Status

Record Version

Scope

SLCS governs:

Lifecycle State Consistency

Lifecycle State Validation

Lifecycle Transition Control

Lifecycle Transition Authorization

Lifecycle Transition Verification

Cross-Standard State Reconciliation

Lifecycle Control Responsibility

Lifecycle Authority Alignment

Lifecycle Control Transfer

Lifecycle Control Continuity

Lifecycle Exception Management

Lifecycle Deviation Recognition

Lifecycle Uncertainty

Lifecycle Conflict Resolution

Lifecycle Re-entry Consistency

Lifecycle Control Evidence

Lifecycle Control Assessment

Lifecycle Control Effectiveness

Lifecycle Reassessment

Lifecycle Governance Continuity

SLCS does not establish universal technical requirements for every software system.

Its requirements apply according to the governed software object, actual

lifecycle conditions, operational significance, and applicable specialized governance.

Foundational Principle

A software lifecycle remains governed only when its declared states, actual operational conditions, authorized transitions, responsible actors, and supporting evidence remain consistent throughout the lifecycle.

A lifecycle state must not be considered legitimate solely because it appears in a software management system.

A transition must not be considered authorized solely because a technical mechanism permits it.

And control responsibility must not be considered effective solely because an organizational role has been assigned.

Declared Lifecycle State ≠ Verified Lifecycle State.

Technical Transition Capability ≠ Legitimate Transition Authority.

Assigned Control ≠ Effective Control.

Software Lifecycle Identity

SLCS requires lifecycle governance to remain connected to an identifiable

software object.

Relevant elements include:

Software Object ID

Product Identity

Version

Build

Artifact Reference

Configuration

Deployment Environment

Operational Instance

Ownership Reference

Lifecycle Control Reference

A product-level lifecycle state must not automatically be applied to every version, configuration, or deployment instance.

Lifecycle Identity Continuity

Software identity must remain traceable across material lifecycle changes.

Potential changes include:

Version Evolution

Configuration Modification

Build Replacement

Environment Transition

Ownership Transfer

Operational Responsibility Transfer

Software Migration

Partial Retirement

Complete Retirement

Identity continuity must preserve the relationship between earlier and later software states without treating materially different objects as identical.

Lifecycle State

A lifecycle state represents an identifiable condition of a software object

within its governed lifecycle.

Potential states include:

Requirements Defined

Creation Active

Development Completed

Dependency Assessment Pending

Configuration Established

Quality Assessment Active

Security Assessment Active

Release Candidate

Release Authorized

Deployment Active

Operational

Change Active

Restricted

Unsupported

Retirement Pending

Partially Retired

Retired

Unknown

These are illustrative lifecycle conditions rather than a mandatory universal state sequence.

The applicable state model must reflect the governed software reality.

Lifecycle State Dimensions

A software object may possess multiple lifecycle dimensions simultaneously.

Relevant dimensions include:

Development State

Configuration State

Quality State

Security State

Release State

Deployment State

Operational State

Support State

Change State

Retirement State

SLCS must preserve these dimensions without forcing them into an inaccurate single status.

Declared Lifecycle State

The declared lifecycle state is the status represented by an applicable organizational or technical governance record.

Potential sources include:

Software Inventory

Development Platform

Release Management System

Deployment Platform

Service Management System

Configuration Register

Retirement Register

A declared state is a governance claim requiring appropriate evidentiary support.

Actual Lifecycle State

The actual lifecycle state represents the software's established technical

and operational condition.

Relevant elements include:

Existing Artifacts

Current Configuration

Deployment Presence

Runtime Execution

Operational Use

Active Dependencies

Available Support

Applicable Restrictions

Continuing Responsibilities

Actual state assessment must account for the software object and environment to which the conclusion applies.

Lifecycle State Consistency

SLCS must establish whether material lifecycle state claims remain mutually consistent.

Consistency assessment may concern:

Software Identity

Version

Configuration

Environment

Release Status

Deployment Status

Operational Status

Support Status

Retirement Status

Supporting Evidence

Different lifecycle dimensions may legitimately have different states.

SLCS must distinguish legitimate multidimensional conditions from contradictory claims.

Lifecycle State Conflict

A lifecycle state conflict exists when material state claims cannot be reconciled under the applicable governance conditions.

Potential conflicts include:

Software Declared Retired While Still Operational

Software Declared Released Without Applicable Release Authorization

Software Declared Secure Despite Unresolved Mandatory Security Conditions

Software Declared Withdrawn While Active Deployments Remain

Software Declared Unsupported While Verified Support Capabilities Remain

Conflicts must be identified, assessed, and resolved or explicitly preserved as unresolved.

Lifecycle State Verification

SLCS must require appropriate evidence for material lifecycle state conclusions.

Potential evidence includes:

Artifact Records

Configuration Baselines

Quality Assessments

Security Assessments

Release Decisions

Deployment Records

Operational Monitoring

Change Records

Retirement Verification

Evidence must remain applicable to the software version, configuration, environment, and time being assessed.

Lifecycle Transition

A lifecycle transition is a material change between identifiable lifecycle conditions.

Potential transitions include:

Requirements to Creation

Creation to Validation

Validation to Release

Release to Deployment

Deployment to Operation

Operation to Change

Operation to Restriction

Operation to Retirement

Retirement Pending to Verified Retirement

Transitions may occur across multiple dimensions and do not necessarily follow a single linear sequence.

Lifecycle Transition Identity

Every material governed transition must remain identifiable.

Relevant elements include:

Transition ID

Software Object ID

Source State

Intended Destination State

Transition Purpose

Applicable Governance Requirements

Responsible Actor

Transition Authority

Execution Evidence

Verification Evidence

Resulting State

Lifecycle Transition Conditions

SLCS must establish the conditions applicable to a proposed transition.

Relevant elements include:

Source State Validity

Destination State Requirements

Applicable Parent Standards

Mandatory Preconditions

Required Evidence

Competent Authority

Operational Constraints

Known Dependencies

Verification Requirements

A transition must not be considered legitimate merely because the destination state can be technically reached.

Lifecycle Transition Authority

SLCS must identify authority for material lifecycle transitions.

Relevant elements include:

Decision Authority

Execution Authority

Verification Authority

Exception Authority

Delegation Conditions

Authority Scope

Authority Validity

External Approval Requirements

Authority for one lifecycle transition does not automatically establish authority for another.

Transition Preconditions

Mandatory transition preconditions must be evaluated before the applicable action.

Potential conditions include:

Required Assessment Completed

Required Configuration Established

Required Quality Evidence Available

Required Security Conditions Addressed

Applicable Release Authorization

Deployment Readiness

Operational Continuity

Retirement Readiness

The actual requirements depend on the transition and the specialized Parent

Standards involved.

Transition Execution

SLCS must preserve the governance relationship between authorized transition conditions and actual execution.

Relevant elements include:

Execution Actor

Execution Mechanism

Applicable Authority

Source State

Execution Time

Observed Change

Resulting State

Execution Evidence

Detailed execution governance remains within the applicable specialized lifecycle standard.

Transition Verification

A transition must be verified according to its intended result.

Relevant elements include:

Intended Destination State

Observed Destination State

Applicable Verification Criteria

Supporting Evidence

Known Deviations

Unresolved Conditions

Verification Responsibility

Resulting Classification

Execution completion must remain distinguishable from verified state achievement.

Failed Lifecycle Transition

A transition may fail when the intended state is not achieved or mandatory governance conditions are violated.

Relevant elements include:

Failed Transition Identity

Source State

Intended State

Actual State

Failure Condition

Operational Consequences

Required Response

Reassessment Conditions

Failure must not automatically be recorded as successful completion merely because the transition activity has ended.

Partial Lifecycle Transition

A transition may be completed for only part of its intended scope.

Potential conditions include:

Partial Deployment

Partial Migration

Partial Configuration Change

Partial User Transition

Partial Service Withdrawal

Partial Retirement

SLCS must preserve the completed and incomplete portions separately.

Lifecycle Re-entry

Software may require renewed governance activity in an earlier lifecycle domain.

Potential triggers include:

Requirements Change

Dependency Change

Security Finding

Quality Defect

Configuration Modification

Operational Incident

Migration Requirement

Retirement Deviation

SLCS governs consistency of lifecycle states and control relationships

during re-entry.

Detailed re-entry requirements remain governed by the applicable

specialized mechanism, including SCMS.

Nonlinear Lifecycle Governance

SLCS recognizes that software lifecycle governance may involve:

Parallel Activities

Repeated Assessments

Conditional Transitions

Multiple Active Versions

Concurrent Environments

Lifecycle Re-entry

Partial Withdrawal

Temporary Restriction

Continuing Post-Retirement Obligations

The lifecycle model must reflect these conditions rather than impose an artificial linear sequence.

Cross-Standard Governance Consistency

SLCS must establish consistency between material governance conclusions produced by different Parent Standards.

Relevant relationships include:

Requirements and Creation

Dependencies and Configuration

Configuration and Quality

Quality and Security

Security and Release

Release and Deployment

Deployment and Change Management

Change Management and Retirement

A valid conclusion within one standard must not automatically override unresolved mandatory conditions in another.

Cross-Standard Evidence Continuity

Evidence produced in one lifecycle domain may be relevant to another.

SLCS must preserve:

Evidence Origin

Software Identity

Applicable Scope

Version and Configuration

Verification Method

Evidence Time

Known Limitations

Receiving Governance Context

Evidence reuse must remain subject to applicability and validity.

Lifecycle Control Responsibility

SLCS must identify responsibility for maintaining lifecycle governance.

Relevant functions include:

Lifecycle Control Owner

Software Owner

Operational Owner

Configuration Responsible Function

Quality Responsible Function

Security Responsible Function

Release Responsible Function

Deployment Responsible Function

Change Responsible Function

Retirement Responsible Function

Responsibility must remain distinguishable from authority and technical capability.

Lifecycle Control Authority

SLCS must preserve the authority structure applicable to lifecycle control.

Relevant elements include:

Authority Origin

Authority Holder

Authority Scope

Delegation

Permission

Decision Boundary

Execution Boundary

Override Conditions

Validity Period

Revocation Conditions

Lifecycle authority must not be inferred solely from system access.

Responsibility–Authority–Capability Alignment

Effective lifecycle control requires appropriate alignment between:

Assigned Responsibility

Legitimate Authority

Operational Capability

Available Resources

Actual Control Conditions

Verification Capability

A function assigned responsibility for software governance may lack the authority or technical capability to enforce the required controls.

SLCS must identify such misalignment rather than treating organizational assignment as proof of effective control.

Lifecycle Control Transfer

Lifecycle control may transfer between actors, teams, organizations, systems, or service providers.

Relevant elements include:

Previous Control Holder

Successor Control Holder

Affected Software

Transferred Responsibility

Transferred Authority

Required Capability

Control Evidence

Effective Transfer Condition

Continuing Obligations

Control transfer must be verified rather than assumed from organizational change.

Lifecycle Control Continuity

SLCS must preserve control continuity during:

Personnel Changes

Organizational Restructuring

Vendor Changes

Outsourcing

Platform Migration

Software Ownership Transfer

Automation Changes

Autonomous System Replacement

Retirement

A lifecycle control transition must not create an unrecognized interval without competent responsibility or authority.

Control Transfer Failure

A control transfer may fail when:

Successor Responsibility Is Unclear

Required Authority Is Missing

Technical Access Is Unavailable

Operational Knowledge Is Lost

Required Evidence Is Inaccessible

Control Capability Is Insufficient

Continuing Obligations Are Unassigned

Material failure must trigger reassessment and appropriate corrective governance.

Lifecycle Exception

A lifecycle exception is a governed departure from an otherwise applicable lifecycle requirement.

Relevant elements include:

Exception Identity

Affected Software

Applicable Requirement

Reason

Competent Authority

Alternative Control

Residual Risk

Validity Conditions

Review Requirements

An exception must not be treated as the disappearance of the original requirement.

Exception Legitimacy

SLCS must evaluate whether a lifecycle exception is legitimate.

Relevant considerations include:

Authority

Justification

Scope

Necessity

Proportionality

Operational Consequences

Alternative Controls

Evidence

Duration or Validity Conditions

An exception cannot create authority that the approving actor does not possess.

Exception Continuity

Exceptions may remain relevant across lifecycle transitions.

SLCS must establish whether an exception:

Remains Applicable

Requires Reassessment

Expires

Transfers

Is Superseded

Must Be Withdrawn

Requires Escalation

An exception approved for one software state must not automatically remain valid after material changes.

Lifecycle Deviation

A lifecycle deviation occurs when the actual lifecycle condition or activity differs from the applicable governed requirements.

Potential deviations include:

Unauthorized Transition

Missing Verification

Unapproved Configuration

Unresolved Security Condition

Invalid Lifecycle State

Incomplete Control Transfer

Expired Exception

Unsupported Retirement Claim

Deviations must remain distinguishable from authorized exceptions.

Lifecycle Uncertainty

SLCS must preserve uncertainty affecting lifecycle governance.

Potential conditions include:

Unknown Software State

Unverified Deployment

Conflicting Lifecycle Records

Unclear Transition Authority

Missing Verification Evidence

Unknown Responsible Function

Incomplete Control Transfer

Unverified Retirement

Material uncertainty must not be converted into a definitive compliant lifecycle status without evidence.

Lifecycle Control Conflict

A lifecycle control conflict may arise when multiple actors or systems assert incompatible control over the same governed software condition.

Relevant elements include:

Conflicting Actors

Affected Software

Conflicting Decisions

Authority Claims

Applicable Requirements

Operational Consequences

Evidence

Resolution Responsibility

Conflict resolution must follow applicable authority and governance boundaries.

Automated Lifecycle Control

Automated systems may perform lifecycle governance activities within authorized boundaries.

Potential activities include:

State Monitoring

Evidence Collection

Transition Validation

Policy Checking

Configuration Verification

Exception Detection

Control Assessment

Governance Reporting

Automation must preserve the distinction between detection, validation, recommendation, authorization, execution, and verification.

Autonomous Lifecycle Actions

Autonomous systems may initiate or execute software lifecycle activities.

SLCS must preserve:

Autonomous Actor Identity

Assigned Objective

Applicable Authority

Permitted Actions

Tool Access

Execution Path

Operational Constraints

Human Override

Evidence

Accountability

Authority for a lifecycle objective does not automatically authorize every tool call or downstream action used to achieve it.

Task Authority ≠ Constituent Action Authority.

Human Oversight and Intervention

Where required by the governed software context, SLCS must establish appropriate human oversight.

Relevant elements include:

Oversight Responsibility

Intervention Authority

Escalation Conditions

Override Capability

Evidence Access

Operational Awareness

Control Transfer Conditions

Intervention Verification

Human oversight must represent an effective control capability rather than a nominal organizational assignment.

Lifecycle Control Evidence

SLCS must preserve evidence supporting material lifecycle control conclusions.

Potential evidence includes:

Software Identity Records

Lifecycle State Records

Transition Decisions

Authority Records

Verification Results

Exception Records

Control Transfer Records

Operational Monitoring

Assessment Findings

Corrective Action Records

Evidence must remain traceable to the governed software and applicable lifecycle conditions.

Lifecycle Auditability

Lifecycle control must support reconstruction of material governance events.

Relevant elements include:

What Software Was Governed

What State Was Claimed

What State Was Verified

What Transition Occurred

Who or What Acted

Under What Authority

What Evidence Was Used

What Exceptions Applied

What Outcome Was Established

Auditability must preserve meaningful governance relationships rather than merely a chronological event log.

Lifecycle Control Assessment

SLCS must establish whether lifecycle governance controls remain appropriate and effective.

Assessment dimensions include:

State Consistency

Transition Legitimacy

Authority Alignment

Responsibility Continuity

Exception Validity

Evidence Sufficiency

Operational Control Capability

Cross-Standard Consistency

Unresolved Governance Conditions

Assessment must distinguish control existence from control effectiveness.

Control Design Assessment

Control design assessment evaluates whether the established governance

mechanism is suitable for the relevant lifecycle condition.

Relevant elements include:

Governed Requirement

Control Purpose

Control Mechanism

Applicable Scope

Responsible Function

Required Authority

Expected Evidence

Known Limitations

A documented control may be unsuitable for the actual software environment.

Control Operational Assessment

Operational assessment evaluates whether the control functions under actual

conditions.

Relevant elements include:

Control Execution

Operational Availability

Actor Capability

Evidence Quality

Observed Results

Failures

Exceptions

Intervention Capability

Continuity

A well-designed control must not be classified as effective solely because it exists in documentation.

Lifecycle Control Deficiency

A control deficiency exists when a required lifecycle control is absent, unsuitable, ineffective, unauthorized, or insufficiently evidenced.

Potential deficiencies include:

Missing State Verification

Unauthorized Transition

Inadequate Exception Control

Unassigned Responsibility

Insufficient Control Capability

Incomplete Authority Transfer

Conflicting Lifecycle Records

Unverified Retirement State

Deficiencies must remain identifiable until appropriately resolved or otherwise governed.

Control Deficiency Significance

SLCS must assess the significance of material deficiencies.

Relevant dimensions include:

Affected Software

Operational Criticality

Lifecycle Stage

Security Implications

Authority Implications

Dependency Consequences

Potential Harm

Evidence Uncertainty

Duration

Significance must reflect actual consequences rather than administrative severity labels alone.

Lifecycle Corrective Governance

Where lifecycle control deficiencies are identified, SLCS must establish appropriate corrective requirements.

Potential actions include:

State Reconciliation

Transition Reassessment

Authority Correction

Responsibility Assignment

Exception Review

Control Transfer Completion

Evidence Recovery

Specialized Lifecycle Re-entry

Control Redesign

Corrective execution remains subject to applicable specialized governance

and authority.

Lifecycle Control Reassessment

SLCS must support reassessment when material conditions change.

Potential triggers include:

Software Version Change

Configuration Change

Deployment Change

New Operational Dependency

Security Finding

Authority Change

Control Responsibility Transfer

Exception Expiration

Lifecycle State Conflict

Retirement Reopening

Reassessment must preserve historical decisions and their applicable evidence.

Lifecycle Control Classification

SLCS recognizes the following governance classifications:

Lifecycle Control Established

Lifecycle Control Active

Lifecycle State Verified

Lifecycle Transition Pending

Lifecycle Transition Authorized

Lifecycle Transition Verification Pending

Lifecycle Transition Verified

Lifecycle Exception Active

Lifecycle Deviation Identified

Lifecycle State Conflict

Control Transfer Pending

Control Continuity Verified

Control Deficiency Identified

Corrective Governance Active

Lifecycle Control Conditionally Valid

Lifecycle Control Not Validated

Reassessment Required

Unknown

Conflicted

Classifications must remain associated with their applicable scope, evidence, and time.

Governance Requirements

A conforming SLCS implementation must satisfy the following requirements. Lifecycle Identity and State

• Establish identifiable software lifecycle objects

• Preserve lifecycle identity across material changes

• Identify applicable lifecycle state dimensions

• Distinguish declared and verified lifecycle states

• Establish evidence requirements for material state claims

• Identify lifecycle state conflicts

• Preserve state uncertainty

• Maintain cross-standard state consistency

Lifecycle Transitions

• Identify material lifecycle transitions

• Establish source and intended destination states

• Identify applicable transition requirements

• Establish mandatory preconditions

• Identify competent transition authority

• Preserve execution traceability

• Verify material transition outcomes

• Identify failed or partial transitions

• Preserve lifecycle re-entry relationships

• Prevent technical execution from substituting for authorized transition.

Control Responsibility and Continuity

• Identify lifecycle control responsibility

• Establish applicable control authority

• Assess required control capability

• Identify responsibility–authority–capability misalignment

• Govern material control transfers

• Preserve control continuity

• Identify transfer failures

• Maintain responsibility for continuing obligations

Exceptions and Deviations

• Identify lifecycle exceptions

• Establish exception authority

• Preserve exception justification and scope

• Establish alternative controls where required

• Preserve exception validity conditions

• Identify lifecycle deviations

• Distinguish deviations from authorized exceptions

• Reassess exceptions after material changes

• Preserve unresolved exception conditions

Evidence and Assessment

• Preserve material lifecycle control evidence

• Maintain lifecycle auditability

• Assess control design suitability

• Assess operational control effectiveness

• Identify material control deficiencies

• Establish corrective governance requirements

• Preserve reassessment triggers

• Support automated control within legitimate authority

• Preserve human oversight where required

• Maintain consistency across all applicable SLGA™ Parent Standards

Governance Logic

Software Identity → Applicable Lifecycle Context → Declared Lifecycle

State → Actual Lifecycle Condition → State Evidence → State Consistency Assessment → Applicable Governance Requirements → Transition Requirements → Authority Alignment → Control Responsibility → Control Capability → Governed Transition → Transition Verification → Resulting Lifecycle State → Exception and Deviation Assessment → Control Continuity → Control Assessment → Software Lifecycle Control Governance Profile

Where states are consistent:

Declared State → Evidence Verification → Actual State Alignment →

Validated Lifecycle State

Where states conflict:

Declared State → Conflicting Operational Evidence → State Conflict →

Reconciliation → Updated Governance State

Where a transition is proposed:

Source State → Transition Requirements → Preconditions → Authority →

Execution → Verification → Destination State

Where a transition is unauthorized:

Transition Attempt → Authority Failure → Transition Blocked or Deviation

Identified → Corrective Governance

Where control transfers:

Current Control Holder → Successor Assessment → Responsibility and

Authority Alignment → Transfer → Continuity Verification

Where an exception is required:

Applicable Requirement → Exception Request → Legitimacy Assessment →

Competent Authority → Conditions → Governed Exception

Where a control deficiency is identified:

Control Assessment → Deficiency → Significance → Corrective Requirements →

Reassessment

SLCS must not equate a declared lifecycle state with an evidence-supported and legitimately governed lifecycle condition.

Lifecycle Control Across Parent Standards Software Requirements

Standard (SRS)

SLCS governs consistency of requirements-related lifecycle states and

transitions.

It does not define the software requirements themselves. Software Creation Standard (SCS)

SLCS governs creation-state consistency, applicable transition authority,

and continuity of lifecycle identity.

It does not replace software creation governance. Software Dependency Standard (SDS)

SLCS governs lifecycle implications of dependency conditions and changes.

It does not replace dependency identification, suitability, or risk governance. Software Configuration Standard (SCFS)

SLCS governs consistency between software lifecycle states and

configuration identity.

It does not replace configuration baseline or build governance. Software Quality Standard (SQS)

SLCS governs lifecycle transitions that depend on applicable quality conditions.

It does not establish the specialized quality criteria or testing mechanisms. Software Security Standard (SSS)

SLCS governs lifecycle implications of security conditions, restrictions,

and unresolved requirements.

It does not replace security assessment or remediation. Software Release Standard (SRLS)

SLCS governs consistency between release states, authorization, and

subsequent lifecycle conditions.

It does not replace release readiness or release authorization. Software Deployment Standard (SDPS)

SLCS governs consistency between deployment state, operational state, and

lifecycle transition conditions.

It does not replace deployment execution or production state monitoring. Software Change Management Standard (SCMS)

SLCS governs lifecycle state and control consistency during software

changes and lifecycle re-entry.

It does not replace change request, impact, coordination, re-entry, or change closure mechanisms. Software Retirement Standard (SRTS)

SLCS governs consistency of retirement-related lifecycle states, authority,

and continuing control obligations.

It does not replace retirement planning, support management, migration, residual dependency management, or retirement closure verification.

Cross-Standard Boundary

SLCS establishes a common control layer without absorbing the governed

mechanisms of other Parent Standards.

Its purpose is to determine whether lifecycle states, transitions, authority relationships, control responsibilities, and evidence remain coherent across the architecture.

It does not create a second approval process for every activity already governed by a specialized standard.

Where a specialized standard establishes the competent decision or verification mechanism, SLCS preserves and evaluates its lifecycle implications.

Specialized Lifecycle Governance ≠ Cross-Cutting Lifecycle Control.

Cross-Standard Consistency ≠ Centralized Replacement of Specialized

Authority.

Lifecycle Control Authority Boundary

SLCS does not establish universal authority over all software lifecycle activities.

Authority remains dependent on the governed object, applicable

organization, delegated permissions, contractual conditions, operational responsibilities, and other legitimate authority sources.

SLCS governs whether these authority relationships are identifiable, applicable, continuous, and consistent.

It must not manufacture authority through methodology classification.

Lifecycle Control and Operational Reality

Software lifecycle governance must reflect actual software conditions.

A release may be authorized without being deployed.

A deployment may exist without being operational.

An application may remain operational after support ends.

A software change may be implemented without its intended outcome being verified.

A product may be declared retired while an active instance remains.

SLCS must preserve these distinctions rather than collapse them into a single simplified lifecycle label.

Lifecycle Governance Must Follow Operational Reality.

Lifecycle Control and Evidence

Lifecycle control conclusions must remain evidence-supported.

Evidence must establish the applicable software identity, governed condition, relevant time, verification method, and known limitations.

The absence of evidence must not automatically be interpreted as evidence of compliance.

The presence of a technical record must not automatically establish that the underlying action was legitimate.

Recorded Activity ≠ Authorized Activity.

Recorded State ≠ Verified State.

Lifecycle Control and Accountability

SLCS must preserve the relationship between lifecycle decisions, responsible actors, applicable authority, and resulting operational consequences.

Relevant elements include:

Decision Origin

Responsible Actor

Authority Basis

Applicable Requirements

Execution Responsibility

Verification Responsibility

Evidence

Resulting Condition

Accountability must remain connected to actual control and decision relationships.

Lifecycle Control and Autonomous Systems

Where software lifecycle activities are performed by autonomous systems, SLCS must preserve the same fundamental governance logic applicable to human and organizational actors.

The actor may change.

The requirement for legitimate authority, identifiable responsibility,

bounded execution, verification, and accountability does not disappear.

Autonomous capability must not be treated as an independent source of authority.

Lifecycle Control Continuity After Retirement

Software retirement may terminate an operational lifecycle role without eliminating every governance obligation.

Potential continuing conditions include:

Historical Evidence

Data Retention

Recovery Obligations

Audit Requirements

Incident Investigation

Residual Dependencies

Contractual Responsibilities

Archived Artifacts

SLCS must preserve the applicable governance continuity of these conditions without treating them as evidence that operational retirement has failed.

Governance Outputs

The principal output is:

Software Lifecycle Control Governance Profile

Supporting outputs may include:

Software Lifecycle Identity Record

Lifecycle State Profile

Lifecycle State Consistency Assessment

Lifecycle State Conflict Record

Lifecycle Transition Profile

Lifecycle Transition Authorization Record

Lifecycle Transition Verification Record

Cross-Standard Governance Consistency Record

Lifecycle Control Responsibility Profile

Lifecycle Authority Alignment Record

Lifecycle Control Transfer Profile

Lifecycle Control Continuity Record

Lifecycle Exception Register

Lifecycle Deviation Register

Lifecycle Uncertainty Record

Lifecycle Control Assessment

Lifecycle Control Deficiency Record

Lifecycle Corrective Governance Record

Lifecycle Reassessment Record

Core Modules

SLCS consists of five Core Modules. 67.1 Lifecycle State Consistency Control Module (LSCCM)

Module ID: SLGA-SLCS-LSCCM-01

Governed Space: Software Lifecycle State Consistency Governance

Principal Output: Software Lifecycle State Consistency Profile

LSCCM governs the identification, comparison, validation, and

reconciliation of declared and actual software lifecycle states.

It establishes consistency between software identity, lifecycle dimensions,

operational conditions, and supporting evidence.

It identifies contradictory, incomplete, outdated, or unverified lifecycle

state claims.

Operational Question:

How can the lifecycle state of an identifiable software object remain consistent and verifiable across multiple systems, environments, versions, and governance domains? 67.2 Lifecycle Transition Control Module (LTCM)

Module ID: SLGA-SLCS-LTCM-02

Governed Space: Software Lifecycle Transition Governance

Principal Output: Software Lifecycle Transition Control Profile

LTCM governs the conditions under which software may legitimately transition between identifiable lifecycle states.

It establishes transition identity, source and destination conditions,

mandatory preconditions, applicable authority, execution traceability, and outcome verification.

Operational Question:

How can software lifecycle transitions occur only under legitimate conditions, applicable authority, and verifiable state changes? 67.3 Lifecycle Exception Management Module (LEMM)

Module ID: SLGA-SLCS-LEMM-03

Governed Space: Software Lifecycle Exception Governance

Principal Output: Software Lifecycle Exception Governance Profile

LEMM governs exceptional departures from applicable software lifecycle

requirements.

It establishes exception identity, justification, authority, scope,

alternative controls, validity conditions, residual risk, reassessment, and termination.

Operational Question:

How can exceptions to software lifecycle requirements remain legitimate, bounded, evidence-supported, and accountable without undermining the governance requirements they modify? 67.4 Lifecycle Control Transfer &

Continuity Module (LCTCM)

Module ID: SLGA-SLCS-LCTCM-04

Governed Space: Software Lifecycle Control Transfer and Continuity

Governance

Principal Output: Software Lifecycle Control Continuity Profile

LCTCM governs the transfer and preservation of software lifecycle control

between responsible actors, teams, organizations, systems, and service providers.

It establishes control identity, responsibility, authority, capability,

transfer conditions, continuity verification, and unresolved control obligations.

Operational Question:

How can software lifecycle control remain effective and accountable when responsibility, authority, or operational capability transfers between actors or systems? 67.5 Lifecycle Control Assessment Module (LCAM)

Module ID: SLGA-SLCS-LCAM-05

Governed Space: Software Lifecycle Control Assessment Governance

Principal Output: Software Lifecycle Control Assessment Profile

LCAM governs the evidence-supported assessment of lifecycle control design,

applicability, operational effectiveness, deficiencies, and corrective requirements.

It evaluates whether lifecycle governance mechanisms actually function

under the conditions they are intended to govern.

Operational Question:

How can the existence, suitability, and effectiveness of software lifecycle controls be independently distinguished and verified throughout the software lifecycle?

Core Module Governance Flow

Lifecycle State Consistency Control → Lifecycle Transition Control →

Lifecycle Exception Management → Lifecycle Control Transfer & Continuity → Lifecycle Control Assessment

This flow represents the principal governance relationships between the

five modules.

It does not require every software event to pass through all five modules in a fixed sequence.

Exception management, control transfer, and control assessment may be triggered whenever their governed conditions arise.

The modules remain interconnected through software identity, lifecycle state, authority, evidence, and governance continuity.

Cross-Architecture Boundary

SLCS governs software lifecycle control within SLGA™.

It may interact with broader OOF® governance architectures where their independently governed mechanisms apply.

Relevant relationships include:

Authority Governance

Legitimate authority, delegation, permissions, and control boundaries.

Accountability Governance

Responsibility, control, decision attribution, and accountability

relationships.

Identity Governance

Continuity and verification of software, actor, and system identities.

Operational Reality Governance

Distinguishing declared lifecycle conditions from actual operational states.

AI Governance

Governance of AI-related software behavior and associated requirements.

Autonomous Systems Governance

Authority, execution, intervention, and control of autonomous software

activities.

SLCS does not redefine these governance domains.

It establishes their applicable relationship to software lifecycle control.

Critical Distinctions

Software Identity ≠ Lifecycle State.

Declared Lifecycle State ≠ Verified Lifecycle State.

Lifecycle State Recorded ≠ Lifecycle State Validated.

Lifecycle Activity Completed ≠ Required State Achieved.

Technical Transition Capability ≠ Legitimate Transition Authority.

Transition Requested ≠ Transition Authorized.

Transition Executed ≠ Transition Verified.

Partial Transition ≠ Complete Transition.

Lifecycle Re-entry ≠ Complete Lifecycle Repetition.

Cross-Standard Consistency ≠ Replacement of Specialized Governance.

Control Assigned ≠ Control Effective.

Responsibility Assigned ≠ Authority Established.

Authority Established ≠ Capability Available.

Control Transfer Announced ≠ Control Continuity Verified.

Exception Requested ≠ Exception Authorized.

Authorized Exception ≠ Elimination of the Original Requirement.

Lifecycle Deviation ≠ Legitimate Exception.

Control Documented ≠ Control Operational.

Control Executed ≠ Control Effective.

Automated Action ≠ Authorized Action.

Software Retired Administratively ≠ Operational Retirement Verified.

Operational Retirement ≠ Elimination of Continuing Governance Obligations.

Canonical Boundary

Software Lifecycle Control Standard (SLCS) governs the consistency,

legitimacy, continuity, and effectiveness of software lifecycle states, transitions, exceptions, authority relationships, and control mechanisms across SLGA™.

It establishes a common lifecycle control framework without replacing the

specialized governance of requirements, creation, dependencies, configuration, quality, security, release, deployment, change management, or retirement.

SLCS does not impose a universal linear software lifecycle.

It does not create authority through methodological classification.

It does not treat administrative records as proof of operational reality.

It does not assume that automated execution establishes legitimate permission.

And it does not equate the existence of governance controls with their operational effectiveness.

Its principal governance output is the:

Software Lifecycle Control Governance Profile

This output establishes whether the software's lifecycle state,

transitions, authority, control responsibility, exceptions, and supporting evidence remain coherent and governable throughout the applicable lifecycle.

Canonical Closing Statement

A software lifecycle is not governed merely because its individual stages have governance requirements. It is governed when those requirements remain consistent as the software moves between states, environments, actors, and operational conditions.

Software may satisfy creation requirements while its dependencies remain uncertain.

A release may be authorized while deployment conditions are incomplete.

A deployed system may change without the necessary lifecycle governance being repeated.

A software component may become unsupported while continuing to perform a critical operational function.

A retirement may be declared while active execution paths remain.

And an autonomous system may perform technically valid lifecycle actions without possessing legitimate authority to execute them.

These conditions reveal a governance problem that cannot be resolved by treating each lifecycle stage in isolation.

SLCS establishes the cross-cutting control structure required to address

that problem.

It connects lifecycle states to actual software conditions.

It establishes the legitimacy of transitions.

It preserves responsibility, authority, and control continuity.

It governs exceptions without allowing them to erase underlying

requirements.

It identifies conflicts between declared and operational reality.

It evaluates whether controls function as intended.

And it preserves the coherence of lifecycle governance across every

specialized Parent Standard within SLGA™.

Software lifecycle governance is not simply the governance of individual stages. It is the preservation of legitimate control across every transition between them.

Software Lifecycle Control Standard (SLCS) completes the eleven-standard

structure of SLGA™ by establishing the continuous governance layer that keeps software identifiable, its lifecycle states verifiable, its transitions legitimate, and its control relationships accountable from initial requirements through retirement and continuing obligations.

Module Architecture

→ Lifecycle State Consistency Control Module (LSCCM)

→ Lifecycle Transition Control Module (LTCM)

→ Lifecycle Exception Management Module (LEMM)

→ Lifecycle Control Transfer & Continuity Module (LCTCM)

→ Lifecycle Control Assessment Module (LCAM)

Related Documents

→ About Software Lifecycle Control Standard (SLCS)